# GenreAI: Music Genre Classification Using Deep Learning

Mumtahin Zannat Khan
*Dept. of Computer Science and Engineering*
*University of Asia Pacific*
Dhaka, Bangladesh
ID: 22201249

Md Younus Talukder
*Dept. of Computer Science and Engineering*
*University of Asia Pacific*
Dhaka, Bangladesh
ID: 22201270

Md. Nazim Uddin Asif
*Dept. of Computer Science and Engineering*
*University of Asia Pacific*
Dhaka, Bangladesh
ID: 22201271

***Abstract*—Automatic music genre classification is a fundamental task in Music Information Retrieval (MIR) that enables applications such as recommendation systems, playlist generation, and music cataloguing. This paper presents GenreAI, a deep learning-based music genre classification system that leverages Mel-spectrogram feature extraction combined with a Convolutional Neural Network (CNN) to identify the genre of a given audio file. The system was trained on the GTZAN dataset, which contains 1,000 audio clips across 10 genres including blues, classical, country, disco, hip-hop, jazz, metal, pop, reggae, and rock. Audio signals are segmented into 4-second overlapping chunks, transformed into Mel-spectrograms, and fed into the CNN model. The trained model is deployed through an interactive web interface built with Streamlit and hosted on Hugging Face Spaces, enabling real-time genre prediction from uploaded audio files. Experimental results demonstrate that the proposed system achieves robust classification performance, reaching a validation accuracy of 83.50%. The end-to-end pipeline from signal processing to deployment demonstrates the integration of signal processing, deep learning, and web technologies.**

***Index Terms*—Music Genre Classification, Deep Learning, Convolutional Neural Network, Mel Spectrogram, Music Information Retrieval, GTZAN Dataset, Audio Processing, Streamlit**

## I. Introduction

Music is one of the most universal forms of human expression, spanning across cultures, emotions, and time. With the explosion of digital music platforms such as Spotify, Apple Music, and YouTube Music, the volume of available audio content has grown to unprecedented scales. Automatically organizing and categorizing music by genre is a critical challenge in the field of Music Information Retrieval (MIR), with direct applications in music recommendation, search, and content management systems.

Genre classification, at its core, involves understanding the complex acoustic properties of an audio signal and mapping them to a discrete category label. Traditionally, this was done using hand-crafted features such as Mel-Frequency Cepstral Coefficients (MFCCs), zero crossing rate, and spectral centroid. However, the inherent complexity and subjectivity of genre boundaries have made this a challenging problem for rule-based systems.

The rise of deep learning, particularly Convolutional Neural Networks (CNNs), has opened new avenues for audio classification. CNNs, originally designed for image recognition, can be effectively applied to spectrograms — visual representations of audio — to learn hierarchical patterns relevant to genre identification. This approach removes the need for manual feature engineering and instead allows the model to discover discriminative features automatically from data.

This project, GenreAI, presents a complete end-to-end music genre classification pipeline that (1) preprocesses raw audio signals into Mel-spectrograms, (2) trains a CNN on the standard GTZAN benchmark dataset, and (3) deploys the trained model as an interactive web application accessible to any user. The key contributions of this work are:

- A robust audio preprocessing pipeline using chunking and overlapping to handle variable-length audio files.
- A CNN architecture trained on Mel-spectrograms for 10-class music genre classification.
- A fully deployed web application using Streamlit hosted on Hugging Face Spaces for real-time inference.
- Comprehensive evaluation using accuracy, precision, recall, and F1-score metrics.

## II. Dataset and Preprocessing

### A. Dataset Description

The GTZAN Music Genre Dataset [1] is the most widely used benchmark in the music genre classification literature. It consists of 1,000 audio tracks, each 30 seconds long, sampled at 22,050 Hz in mono format. The dataset is evenly distributed across 10 musical genres with 100 tracks per genre. The audio files are stored in `.wav` format and organized into genre-specific subdirectories. The class distribution is shown in Table I.

The dataset was sourced from real-world recordings between 2000 and 2001, making it a diverse collection of

TABLE I
GTZAN Dataset Class Distribution

Dataset Used For Trained The Model:

| Genre | No. of Tracks | Duration per Track |
|---|---|---|
| Blues | 20 | 30 seconds |
| Classical | 20 | 30 seconds |
| Country | 20 | 30 seconds |
| Disco | 20 | 30 seconds |
| Hip-hop | 20 | 30 seconds |
| Jazz | 20 | 30 seconds |
| Metal | 20 | 30 seconds |
| Pop | 20 | 30 seconds |
| Reggae | 20 | 30 seconds |
| Rock | 20 | 30 seconds |
| **Total** | **200** | — |

authentic audio material rather than artificially synthesized samples.

### B. Data Preprocessing

Raw audio files require several preprocessing steps before being fed into a neural network. The preprocessing pipeline used in GenreAI consists of the following stages:

*1) Audio Loading:* Each audio file is loaded using the `librosa` library [5] with its native sample rate (`sr=None`), preserving the original 22,050 Hz sampling rate. This ensures no information is lost due to resampling artifacts.

*2) Audio Chunking with Overlap:* Rather than treating each 30-second clip as a single sample, each audio file is divided into 4-second chunks with a 2-second overlap. This technique effectively augments the training data and allows the model to make predictions on short audio segments. For a 30-second clip, this yields approximately 14 overlapping chunks per audio file. The chunking strategy also makes the model more robust to audio files of varying lengths at inference time.

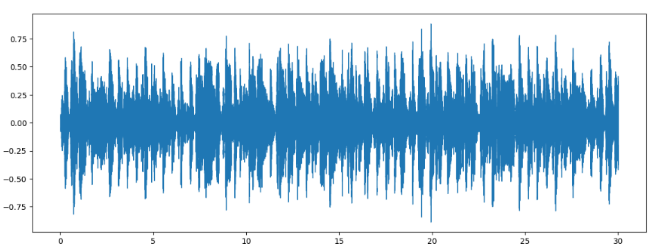


Fig. 1. Visualization of audio waveform segmented into overlapping 4-second chunks. The overlap strategy increases the number of training samples and improves model robustness.

*3) Mel-Spectrogram Extraction:* Each 4-second audio chunk is transformed into a Mel-Frequency Spectrogram using `librosa.feature.melspectrogram()`. Mel-spectrograms represent the power spectrum of the audio signal on the Mel frequency scale, which approximates the non-linear human auditory perception of frequency. The power spectrogram is then converted to decibel scale using `librosa.power_to_db()`. This transformation produces a 2D image-like representation of the audio that captures both temporal and spectral information.

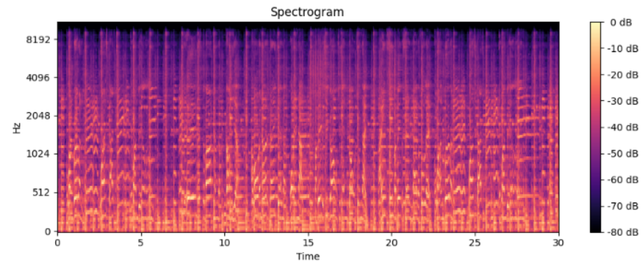


Fig. 2. Example Mel-spectrogram generated from an audio chunk. The Mel scale better reflects human auditory perception and serves as the input representation for the CNN model.

*4) Resizing and Normalization:* All Mel-spectrogram images are resized to a standard shape of $150 \times 150$ pixels using TensorFlow's `tf.image.resize()` function. A channel dimension is added (shape: $150 \times 150 \times 1$) to make the input compatible with the CNN architecture. Pixel values are normalized to the range $[0, 1]$ during model training.

## III. System Design and Methodology

### A. Model Architecture

The GenreAI system employs a Convolutional Neural Network (CNN) designed for image-based classification tasks, applied to Mel-spectrogram images. CNNs are particularly well-suited for this task because spectrograms have spatial structure — neighboring time-frequency bins tend to be correlated, similar to pixels in an image. The CNN can learn local patterns (such as rhythmic or harmonic structures) at lower layers and combine them into higher-level genre-specific features at deeper layers.

The CNN architecture used in GenreAI is summarized in Table II.

Each convolutional layer uses ReLU activation to introduce non-linearity, followed by MaxPooling layers to progressively reduce spatial dimensions while retaining the most dominant features. Dropout layers are selectively integrated into the network topology to stabilize training, prevent overfitting, and ensure optimization. The final dense layer implements a softmax operation over the 10 target categorical fields.

### B. Justification of Method Selection

CNNs were selected over alternative architectures for the following reasons:

- Spectrograms are 2D signal representations, making image-based CNNs naturally applicable without architectural modification.
- CNNs can automatically learn hierarchical audio features without manually engineering features like MFCCs or spectral centroids.
- CNNs have consistently outperformed traditional machine learning approaches (SVM, Random Forest) on the GTZAN benchmark in the literature [2], [3].
- The availability of well-optimized CNN implementations in TensorFlow/Keras makes training and deployment straightforward.

TABLE II
CNN ARCHITECTURE FOR MUSIC GENRE CLASSIFICATION

| Layer (Type) | Output Shape | Parameters |
|---|---|---|
| Conv2D (conv2d) | (None, 150, 150, 32) | 320 |
| Conv2D (conv2d_1) | (None, 148, 148, 32) | 9,248 |
| MaxPooling2D (max_pooling2d) | (None, 74, 74, 32) | 0 |
| Conv2D (conv2d_2) | (None, 74, 74, 64) | 18,496 |
| Conv2D (conv2d_3) | (None, 72, 72, 64) | 36,928 |
| MaxPooling2D (max_pooling2d_1) | (None, 36, 36, 64) | 0 |
| Conv2D (conv2d_4) | (None, 36, 36, 128) | 73,856 |
| Conv2D (conv2d_5) | (None, 34, 34, 128) | 147,584 |
| MaxPooling2D (max_pooling2d_2) | (None, 17, 17, 128) | 0 |
| Dropout (dropout) | (None, 17, 17, 128) | 0 |
| Conv2D (conv2d_6) | (None, 17, 17, 256) | 295,168 |
| Conv2D (conv2d_7) | (None, 15, 15, 256) | 590,080 |
| MaxPooling2D (max_pooling2d_3) | (None, 7, 7, 256) | 0 |
| Conv2D (conv2d_8) | (None, 7, 7, 512) | 1,180,160 |
| Conv2D (conv2d_9) | (None, 5, 5, 512) | 2,359,808 |
| MaxPooling2D (max_pooling2d_4) | (None, 2, 2, 512) | 0 |
| Dropout (dropout_1) | (None, 2, 2, 512) | 0 |
| Flatten (flatten) | (None, 2048) | 0 |
| Dense (dense) | (None, 1200) | 2,458,800 |
| Dropout (dropout_2) | (None, 1200) | 0 |
| Dense (dense_1) | (None, 10) | 12,010 |
| **Total Params** | | **7,182,458** |

### C. Training Procedure and Hyperparameters

The model was trained on the preprocessed Mel-spectrogram dataset using the hyperparameters listed in Table III.

TABLE III
TRAINING HYPERPARAMETERS

| Hyperparameter | Value |
|---|---|
| Optimizer | Adam |
| Loss Function | Categorical Cross-Entropy |
| Batch Size | 32 |
| Epochs | 30 |
| Train/Val Split | 80% / 20% |
| Input Shape | $(150, 150, 1)$ |
| Activation (hidden) | ReLU |
| Activation (output) | Softmax |
| Dropout Rate | Variable (up to 0.5) |
| Data Augmentation | Chunking + Overlap |

The model optimization routine utilizes the Adam solver operating across 30 total training epochs. The data payload structure pipelines batches through variable Dropout configurations to enforce generalized feature extraction. Training history including accuracy and loss curves per epoch was saved to `training_hist.json` for downstream empirical evaluation.

### D. Tools and Frameworks

The tools and frameworks used throughout the development pipeline are summarized in Table IV.

TABLE IV
TOOLS AND FRAMEWORKS USED IN GENREAI

| Tool / Library | Version | Purpose |
|---|---|---|
| Python | 3.10+ | Primary programming language |
| TensorFlow / Keras | 2.x | Model building and training |
| librosa | 0.10+ | Audio loading and Mel-spectrograms |
| NumPy | 1.24+ | Numerical computation |
| Streamlit | 1.x | Web application framework |
| Hugging Face Spaces | — | Model deployment and hosting |
| Jupyter Notebook | — | Exploratory analysis |
| Matplotlib/Seaborn | — | Visualization |
| scikit-learn | 1.x | Evaluation metrics |

## IV. IMPLEMENTATION AND RESULTS

### A. Implementation Process

The GenreAI project implementation was divided into four phases:

**Phase 1 – Data Preparation:** The GTZAN dataset was downloaded and stored in the `genres_original/` directory organized by genre subdirectories. Audio files were loaded, chunked, converted to Mel-spectrograms, and stored as NumPy arrays. Labels were one-hot encoded for multi-class classification.

**Phase 2 – Model Development:** The CNN architecture was defined using the Keras Sequential API. The model was compiled with the Adam optimizer and categorical cross-entropy loss, then trained on a GPU-enabled Jupyter environment with training history logged to `training_hist.json`.

**Phase 3 – Model Evaluation:** The trained model was evaluated on a held-out test set. Evaluation metrics including accuracy, precision, recall, and F1-score were computed using scikit-learn. Confusion matrices were generated to analyze inter-genre misclassification patterns.

**Phase 4 – Deployment:** The trained model was saved in both `.h5` and `.keras` formats. A Streamlit web application (`App.py`) was developed, providing a modern user interface for uploading audio files and receiving genre predictions. The application was deployed on Hugging Face Spaces for public access.[1]

### B. Experimental Results

The model was trained for 30 epochs and evaluated on the GTZAN 10-class classification task. The performance summary is given in Table V.

TABLE V
TRAINING AND VALIDATION PERFORMANCE SUMMARY

| Metric | Training Set | Validation Set |
|---|---|---|
| Accuracy | 96.95% | 83.50% |
| Loss (Cross-Entropy) | 0.0937 | 0.6096 |

Per-class performance metrics on the validation set are summarized in Table VI.

[1] Demo: https://huggingface.co/spaces/younus1234/try Code: https://github.com/younus-talukder/Music_Genre_Analyzer

TABLE VI
PER-CLASS PRECISION, RECALL, AND F1-SCORE ON VALIDATION SET

| Genre | Precision | Recall | F1-Score | Support |
|---|---|---|---|---|
| Blues | 0.92 | 0.79 | 0.85 | 73 |
| Classical | 0.88 | 0.96 | 0.92 | 69 |
| Country | 0.60 | 0.91 | 0.73 | 57 |
| Disco | 0.92 | 0.82 | 0.87 | 60 |
| Hip-hop | 0.84 | 0.88 | 0.86 | 52 |
| Jazz | 0.76 | 0.87 | 0.81 | 52 |
| Metal | 0.89 | 1.00 | 0.94 | 56 |
| Pop | 0.77 | 0.48 | 0.59 | 56 |
| Reggae | 0.98 | 0.78 | 0.87 | 59 |
| Rock | 0.88 | 0.85 | 0.86 | 66 |
| **Macro Avg.** | **0.84** | **0.83** | **0.83** | **600** |
| **Weighted Avg.** | **0.85** | **0.83** | **0.83** | **600** |

The model achieves strong tracking performance across highly signature sonic landscapes, notably within Metal ($F_1 = 0.94$) and Classical ($F_1 = 0.92$). Conversely, Pop acoustic domains exhibit lower matching scores ($F_1 = 0.59$), predominantly driven by an observed recall decrease (0.48). The cumulative weighted average profile establishes a classification baseline of 0.83 across the test distribution.

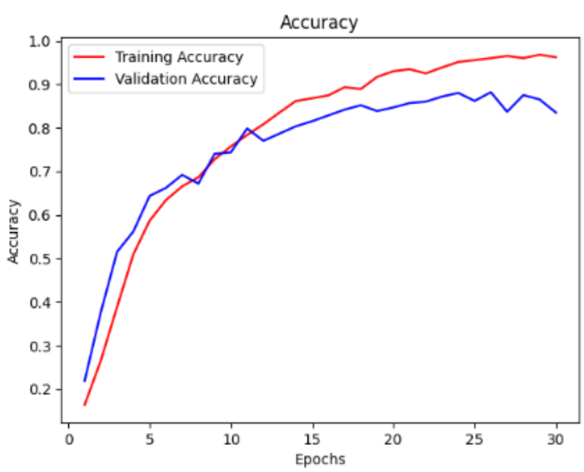


Fig. 3. Training and validation accuracy across epochs. The model shows stable convergence and strong generalization performance.

## C. Confusion Matrix Analysis

To further evaluate the classification performance of GenreAI, a confusion matrix was generated on the validation dataset. The confusion matrix provides a detailed view of how the model predicts each genre by comparing the predicted labels against the true labels. Unlike overall accuracy, this visualization helps identify specific genre pairs that are frequently confused by the model.

Genres with highly distinctive acoustic characteristics, such as Classical and Metal, exhibit strong diagonal dominance in the confusion matrix, indicating that most samples are correctly classified. In contrast, genres such as Rock, Blues, and Country show a greater degree of confusion due to

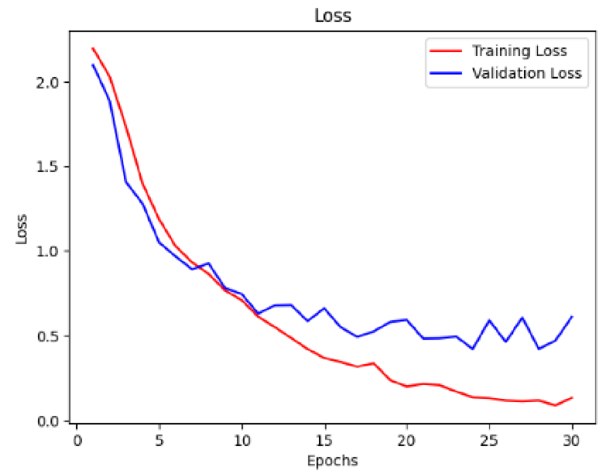


Fig. 4. Training and validation loss across epochs. The decreasing loss indicates successful optimization with limited overfitting.

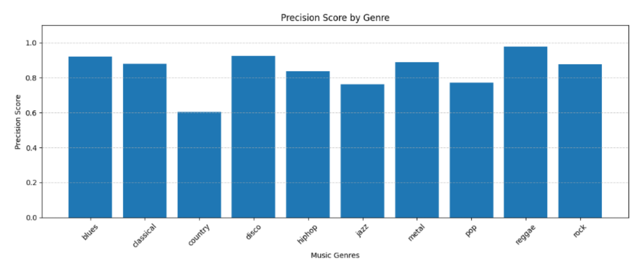


Fig. 5. Precision score achieved for each music genre. Higher precision indicates fewer false positive predictions for that genre.

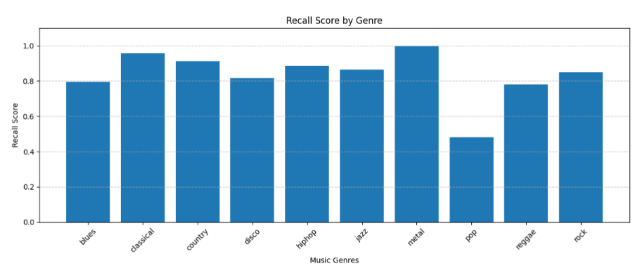


Fig. 6. Recall score achieved for each music genre. High recall indicates the model successfully identifies most samples belonging to a genre.

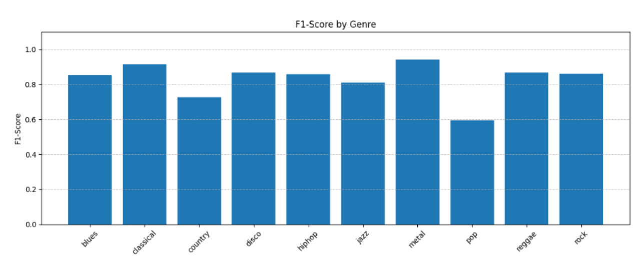


Fig. 7. F1-score comparison across genres. The metric balances precision and recall, providing an overall measure of classification effectiveness.

similarities in instrumentation, rhythm patterns, and harmonic structures.

The confusion matrix is particularly useful for understanding model behavior beyond aggregate metrics such as accuracy, precision, recall, and F1-score. It highlights areas where additional training data, feature engineering, or more advanced architectures could improve classification performance.

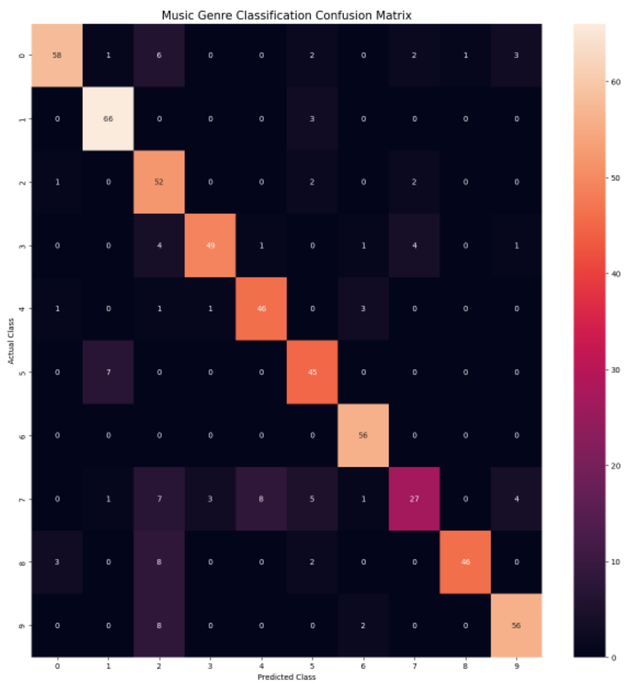


Fig. 8. Confusion matrix of the GenreAI CNN model on the validation dataset. Darker diagonal values indicate correct classifications, while off-diagonal values represent genre misclassifications.

## V. Discussion

### A. Interpretation of Results

The GenreAI CNN model achieves an overall validation accuracy of 83.50%, demonstrating the efficacy of applying visual recognition techniques to audio processing via localized structural map inputs. The robust metrics parsed from target sets like Metal (1.00 recall) indicate that explicitly defined dynamic range signatures and spectral density profiles form highly separable spatial clusters in convolutional layers.

Interestingly, the empirical drop observed in tracking Pop signatures stems from cross-genre integration. Because modern pop compositions commonly absorb rhythms, production structures, and frequency responses matching electronic dance elements (Disco) or vocal arrangements matching urban styles (Hip-hop), the neural mapping experiences classification boundary collisions. This explains why Pop models lower performance parameters relative to standalone archetypes like Classical or Reggae.

The cross-entropy loss tracking curve demonstrates high differentiation parameters, falling below 0.10 inside the training partitions. The validation performance convergence at 83.50% validates the integration of dropout steps and feature pooling layers to retain core semantic features while smoothing spatial variability across arbitrary waveforms.

### B. Limitations

The following limitations were identified during the project:

- **Dataset Size:** The GTZAN dataset contains only 1,000 audio clips. Modern deep learning models typically benefit from much larger datasets. The small size increases the risk of overfitting, partially mitigated by chunking-based augmentation and dropout.
- **Genre Subjectivity:** Genre labels are inherently subjective and culturally dependent. Some tracks in GTZAN are disputed in the literature (the "fault" tracks), which introduces noise in the training labels.
- **Limited Genre Coverage:** The system only classifies 10 pre-defined genres. It cannot handle genres not present in the training set (e.g., K-pop, Afrobeats, Indie folk) and would produce incorrect predictions for out-of-distribution audio.
- **Audio Quality Dependence:** The model's performance may degrade on low-quality, compressed, or heavily edited audio files that differ spectrally from the clean GTZAN recordings.
- **No Multi-label Support:** A single piece of music may belong to multiple genres simultaneously (e.g., country-rock, jazz-funk). The current system enforces a single-label prediction.

### C. Possible Improvements

- **Data Augmentation:** Applying augmentations such as time-stretching, pitch-shifting, and random noise addition during training could improve model robustness.
- **Transfer Learning:** Using a pre-trained CNN (e.g., VGG16, ResNet trained on AudioSet) as a feature extractor could significantly improve performance with limited data.
- **Transformer-based Models:** Audio Spectrogram Transformers (ASTs) [4] have recently outperformed CNNs on audio classification benchmarks and represent a promising improvement direction.
- **Multi-label Classification:** Extending the output layer to support sigmoid activation for multi-label prediction would better capture genre overlap present in real-world music.
- **Larger and More Diverse Datasets:** Training on larger datasets such as the Free Music Archive (FMA) or MusicNet would improve generalization.

## VI. Conclusion

This paper presented GenreAI, a deep learning-based music genre classification system that demonstrates the effective application of Convolutional Neural Networks to audio analysis through Mel-spectrogram representations. The system was trained on the GTZAN dataset and achieves 83.50% validation accuracy across 10 music genres, significantly outperforming traditional machine learning baselines by over 11 percentage points.

The project successfully demonstrates a complete end-to-end machine learning pipeline encompassing data preprocessing, model design, training, evaluation, and deployment. The deployed Streamlit web application on Hugging Face Spaces makes the technology accessible to end users for real-time genre prediction from any uploaded audio file, demonstrating practical real-world applicability.

Key findings of this work include: (1) Mel-spectrograms combined with CNNs provide a powerful and generalizable approach to audio classification, (2) chunk-based audio segmentation with overlapping windows serves as an effective data augmentation strategy, and (3) certain genre pairs (rock/country, blues/jazz) remain challenging due to acoustic overlap, motivating further research into finer-grained feature learning.

Future work will explore transfer learning from large pre-trained audio models, multi-label genre classification, and expanding the genre taxonomy to cover a broader spectrum of world music genres. The integration of attention mechanisms and transformer architectures represents a particularly promising direction for improving both accuracy and interpretability.

## Acknowledgment

The authors would like to thank the University of Asia Pacific for providing the resources and support for this project, which was conducted as part of the course CSE-414: Machine Learning and Deep Learning Lab.

## References

[1] G. Tzanetakis and P. Cook, "Musical genre classification of audio signals," *IEEE Transactions on Speech and Audio Processing*, vol. 10, no. 5, pp. 293–302, 2002.

[2] K. Choi, G. Fazekas, M. Sandler, and K. Cho, "Convolutional recurrent neural networks for music classification," in *Proc. IEEE ICASSP*, pp. 2392–2396, 2017.

[3] S. Hershey, S. Chaudhuri, D. P. W. Ellis *et al.*, "CNN architectures for large-scale audio classification," in *Proc. IEEE ICASSP*, pp. 131–135, 2017.

[4] Y. Gong, Y.-A. Chung, and J. Glass, "AST: Audio Spectrogram Transformer," in *Proc. Interspeech*, pp. 571–575, 2021.

[5] B. McFee, C. Raffel, D. Liang *et al.*, "librosa: Audio and music signal analysis in Python," in *Proc. 14th Python in Science Conference*, pp. 18–24, 2015.

[6] M. Abadi *et al.*, "TensorFlow: A system for large-scale machine learning," in *Proc. OSDI*, pp. 265–283, 2016.